

이벤트 시스템 — Stage 1A-2 검증 결과

날짜: 2026-04-30 결과: ✔ PASS — 1A-2 신규 25/25, 1A 회귀 23/23

선행: 이벤트 시스템 - 1A 검증 결과.md, 이벤트 시스템 구현 - 안 4 (kind 분리 + chain).md

§1. 단계 정의

Stage 1A-2 — run_start 트리거 + 다중 이벤트. 1A 의 단일 이벤트 (node_enter) 를 다중 이벤트로 확장. 새 트리거 타입 run_start 도입.

두 이벤트

id	트리거	라인	once_per
intro_speech	run_start	"시작합니다."	big_run
regression_speech	node_enter, type "event" (노드 5)	"회귀합니다."	big_run

UI 미구현 단계 — 콘솔 검증만, event_state JSON 스키마는 lifecycle 시점마다 print.

§2. 산출물

신규 파일

- test/test_event_1a2.gd / test_event_1a2.tscn — 다중 이벤트 시나리오 검증 (25 체크).

변경 파일

- event_manager.gd
- _resolve_event_for_node → 일반화된 resolve_event(trigger_type, context) + 노드 wrapper.
- match trigger_type 으로 트리거별 매치 조건 분기 (node_enter / run_start).
- game_data.gd::EVENTS
- intro_monologue 폐기 → intro_speech (run_start) + regression_speech (node_enter) 두 개로 교체.
- run_manager.gd
- _begin_event_phase(event_id) 헬퍼 추가 — phase 전이 + EventManager.begin_event + state_changed.

- `_start_internal_run` 끝에 `run_start` 트리거 평가 + 매치 시 `_begin_event_phase` 호출.
- `move_to_node` 의 type "event" 분기 → `_begin_event_phase` 호출로 정리.
- `test/test_event_1a.gd`
- 이벤트 이름 `intro_monologue` → `regression_speech` 갱신.
- 도입부에 `run_start` 이벤트 (`intro_speech`) 자동 소비 루프 추가 (1A 의 본 시나리오에는 `node_enter` 흐름).
- `regression_speech` 가 1 라인이라 중간 `line_idx` 검증 제거.

§3. 헤드리스 실행 명령

```
GODOT="/Users/js/Desktop/Godot.app/Contents/MacOS/Godot"
PROJ="/Users/js/Desktop/퍼즐던전 고도"

# 신규
"$GODOT" --headless --path "$PROJ" res://test/test_event_1a2.tscn

# 회귀
"$GODOT" --headless --path "$PROJ" res://test/test_event_1a.tscn
```

§4. 1A-2 검증 시나리오 (25 체크)

섹션 1 — `start_run` → `run_start` → `intro_speech` (8)

- [x] `phase = 'event'` (`intro_speech` 자동 발화)
- [x] `event_state.event_id = 'intro_speech'`
- [x] `event_state.kind = 'dialogue'`
- [x] `event_state.line_idx = 0`
- [x] `event_state.chain_root_id = 'intro_speech'`
- [x] intro 종료 후 `event_state = {}`
- [x] intro 종료 후 `phase = 'map'`
- [x] `seen_events[intro_speech] = 1`

섹션 2 — `node 5 진입` → `regression_speech` (9)

- [x] event 노드 (5) move 성공
- [x] `phase = 'event'` 전이
- [x] `event_state.event_id = 'regression_speech'`
- [x] `event_state.kind = 'dialogue'`

- [x] event_state.line_idx = 0
- [x] regression 종료 후 event_state = {}
- [x] regression 종료 후 phase = 'map'
- [x] seen_events[regression_speech] = 1
- [x] seen_events[intro_speech] 도 그대로 = 1

섹션 3 — 회귀 (end_internal_run cleared) → once_per: big_run 유지 (8)

- [x] 회귀 후 seen_events[intro_speech] 유지 = 1
- [x] 회귀 후 seen_events[regression_speech] 유지 = 1
- [x] 회귀 후 phase = 'map' (intro_speech 미발생)
- [x] 회귀 후 event_state 비활성
- [x] 회귀 후 current_node_id = 1
- [x] 회귀 후 node 5 move 성공
- [x] 회귀 후 node 5 진입 시 phase = 'map' (regression 미발생)
- [x] 회귀 후 event_state 비활성 유지

§5. event_state JSON 스키마 출력 (lifecycle 시점별)

```
>>> [start_run 직후] event_state = {
  "chain_root_id": "intro_speech",
  "event_id": "intro_speech",
  "kind": "dialogue",
  "line_idx": 0,
  "phase": "awaiting_input"
}

>>> [intro 종료 후] event_state = {}
>>> [intro 종료 후] seen_events = {"intro_speech": 1}

>>> [node 5 진입 직후] event_state = {
  "chain_root_id": "regression_speech",
  "event_id": "regression_speech",
  "kind": "dialogue",
  "line_idx": 0,
  "phase": "awaiting_input"
}

>>> [regression 종료 후] event_state = {}
>>> [regression 종료 후] seen_events = {
  "intro_speech": 1,
  "regression_speech": 1
}
```

```

}

>>> [회귀 직후 (새 internal run 시작 직후)] event_state = {}
>>> [회귀 직후] seen_events = {
    "intro_speech": 1,
    "regression_speech": 1
}

```

(원 출력은 한 줄 JSON, 가독성 위해 줄바꿈.)

§6. 1A 회귀 테스트 결과

23/23 PASS. 1A-2의 변경이 1A의 핵심 시나리오 (node_enter → dialogue → resolve → once_per 필터 → 회귀 통과)를 깨지 않음을 확인.

```

=== Stage 1A 자동 검증 시작 ===
[PASS] intro 소비 후 phase = 'map'
[PASS] seen_events에 regression_speech 아직 미존재
... (25 줄 중 일부 생략) ...
[PASS] 회귀 후에도 once_per 회피 - 이벤트 미발생
---
총 23 검증 - PASS 23 / FAIL 0
=== PASS ===

```

§7. 자가점검

원칙 / 항목	상태
RunManager 단일 추적점 — seen_events ∈ big_run_data	✓
EventManager 일시 상태만 — event_state 비활성 시 {}	✓
트리거 추가 (run_start)가 기존 (node_enter)와 동일 매치 결	✓
_begin_event_phase 헬퍼로 진입 경로 단일화 (run_start / move_to_node 공용)	✓
Public API만 사용한 자동 검증 (private dict 직접 변이 X)	✓
회귀 누적 — 1A 테스트 부분 갱신 후 보존	✓
event_state JSON 스키마 콘솔 가시화	✓

§8. 다음 단계 제안

지금 상태에서 자연스러운 다음 갈래:

1. **Stage 2A** — **effect kind 디스패처** — 안 4 §9.1. UI 입력 없는 즉시 통과 이벤트. Type 1 = `_apply_*` 단일 경로 검증.
2. **Stage 2B** — **chain (`next` 필드)** — 안 4 §9.2. `dialogue` → `effect` → `dialogue` 같은 시퀀스. `chain_root_id` 단일 카운트 검증.
3. **Stage 1B** — **UI 진입** — 1A·1A-2 모두 시뮬 PASS 이므로 UI 만 없으면 되는 상태.

GO 신호 + 어떤 단계부터 진입할지 알려주세요.